GIT GITHUB

EXPOSE EN COURS DE FINALISATION

GIT

VOUS DEVELOPPEZ DES SOLUTIONS LOGICIELLES ?
VOUS CODEZ A PLUSIEURS DEV UNE SOLUTION LOGICIELLE ?
TOUT LE MONDE Y TRAVAILLE ET PERSONNE NE PEUT GARANTIR :

- SI LA VERSION FINALE EST BIEN LA DERNIERE MISE A JOUR!
- SI ON PEUT REVENIR EN ARRIERE POUR RE-CODER SUIVANT UNE AUTRE DEMARCHE!
- QUE SON COLLEGUE LUI TRANSMETTE CORRECTEMENT SA PARTIE DE CODE DANS LA BONNE VERSION
- Etc…

GIT

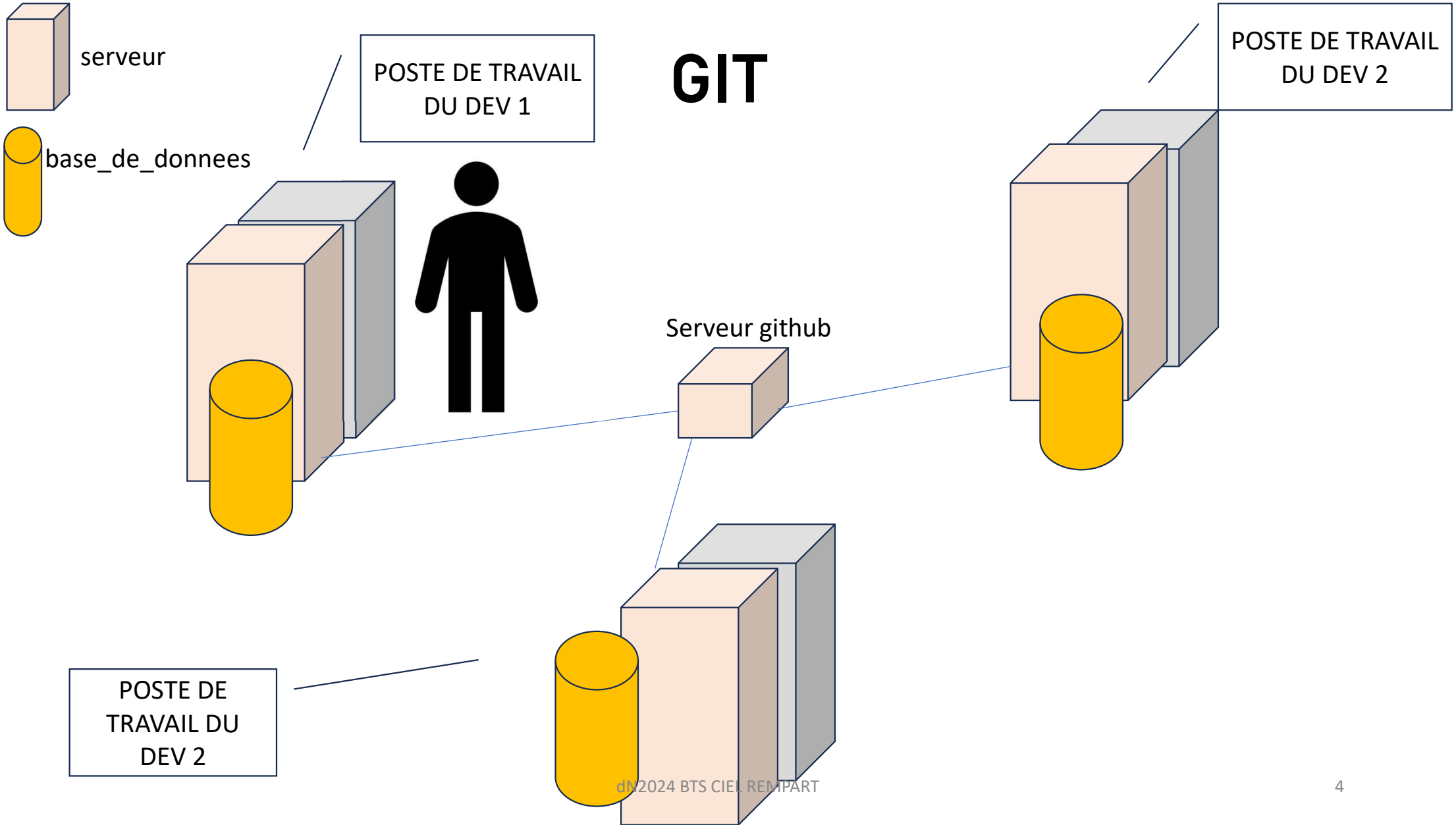
BITMOVER firme us privée a répondu à ce besoin avec BitKeeper mais €€€

Mais mésentente entre l’équipe développant LINUX et BITMOVER !

Linus Torvalds a developpé en 10 jours …

GIT le gestionnaire de code source

GIT


serveur
base_de_donnees
POSTE DE TRAVAIL DU DEV 1
POSTE DE TRAVAIL DU DEV 2
Serveur github
POSTE DE TRAVAIL DU DEV 2

INSTALLER GIT

OUVRIR UN TERMINAL CONSOLE

sudo apt update

sudo apt install git

RE OUVRIR UNE AUTRE FENETRE CONSOLE TERMINAL POUR VERIF OK INSTALL

git --version

CONFIGURER VOTRE CONTRIBUTION
DONNER VOTRE NOM et EMAIL

git config --global user.name ‘’votre nom’’

git config --global user.email ‘’votre@email.com’’

GIT

ORGANISER SON POSTE DE DEV : créer et descendre dans le bon répertoire

sudo mkdir TestRepository

cd TestRepository

VERIFIER L ETAT DU DOSSIER

git status réponse attendue : **fatal:not a git repository**

INITIALISER GIT

git init réponse attendue :
Initialized empty Git repository in [home/etudiant]/TestRempository/.git/

GIT

ETUDE RAPIDE DE CE QUE CREE LES COMMANDES PRECEDENTES

ls -al

vous devez constater « .git » c'est la base de données (vide pour l'instant) répertoriant les modifications successives du code

Ne jamais la commande « git init »

VERIFIER L ETAT DU « REPO » : fichiers en cours d'édition ? Besoin d'enregistrer des fichiers dans git?

git status réponse attendue :

on branch main
No commits yet
Nothing to commit (create/copy files and use « git add » to track)

GIT

UNE « **BRANCHE** » EST UN CODE SOURCE ISOLE DANS UN ESPACE DE TRAVAIL INDEPENDANT.
C EST UNE COPIE PROTEGEE AU SEIN DU MEME DOSSIER TESTREPOSITORY

VERIFIER L ETAT DU « REPO » : fichiers en cours d'édition ? Besoin d'enregistrer des fichiers dans git?

EXPLICATIONS

git status réponse attendue :

on branch main
No commits yet
Nothing to commit (create/copy files and use « git add » to track)

NOUS SOMMES SUR LA **BRANCHE** PRINCIPALE MAIN

PAS ENCORE DE COMMIT
C EST A DIRE AUCUN FICHIER N A ÉTÉ CONFIE A GIT

GIT NE TROUVE AUCUN FICHIER A CONFIER-ENREGISTRER DANS LE « DEPOT-REPO »

UN « **COMMIT** » C'EST METTRE EN CONSIGNE UN FICHIER DANS LE DEPOT

GIT le gestionnaire de code source

MANIPULATION 1:

1_ on crée un fichier à lire : sudo nano README.md

saisir dans ce fichier : « **#ceci est le fichier à lire de notre projet BTS** »

2_ on vérifie l'état du git : git status

réponse attendue : **On branch main…No commits yet…**

Untracked files : README.md

(il est également notifié l'instruction git add <file> pour *committer* le fichier)

3_ établissons **un index** : consignons le fichier README.md :

git add README.md

réponse attendue : **Changes to be commited:…[…] new file : README.md**

4_ enregistrons dans la BDD le code source README (même si ce n'est pas un programme)

git commit –m ''ma premiere version de Readme''

[main (root-commit) xxvvffg] ma premiere version de Readme
1 file changed, 1 insertion
Create mode xffettfg README.md

UNTRACKED FILE signifie « fichier non suivi » par GIT. Git sait simplement qu'il existe des modifs sur les fichiers qu'il référence suite à un commit

Il y a 3 états à comprendre pour comprendre GIT:
État1: **untracked file** (fichier non géré par GIT)
Etat2: **ajouté à l'index** git (un index regroupe l'ensemble des modifications à commiter)
Etat3: **commité** est le fait d'avoir enregistrer dans la BDD de git une étape d'évolution du code source modifié.

GIT le gestionnaire de code source

Suite MANIPULATION 1:

5_ éditer le fichier README.md et modifier le contenu en rajoutant :
#ceci est une modification du readme.md

6_ saisir : git status
Réponse : modified : README.md

7_ saisir : git diff README.md
Réponse : git vous affiche l'ancien et le nouveau contenu

8_ préparons le commit : git add ./README.md

9_ réalisons le commit : git commit –m ''rajout de une ligne''
Réponse : 1 file changed, 1 insertion (+)

L'arbre du dépôt est donc :

GIT le gestionnaire de code source

UNE BRANCHE = 1 copie protégée = 1 version du code

Manipulation 2:

10_ saisir : git status → nous sommes donc toujours sur la branche main ?

11_ nous voulons développer une nouvelle fonctionnalité « feature1 ».
Pour développer cette fonctionnalité, on crée une version de notre code qui va servir
À développer cette fonctionnalité, sans interférence avec d'autres développeurs.
Cette copie protégée c'est la … branche !! Créons la branche :

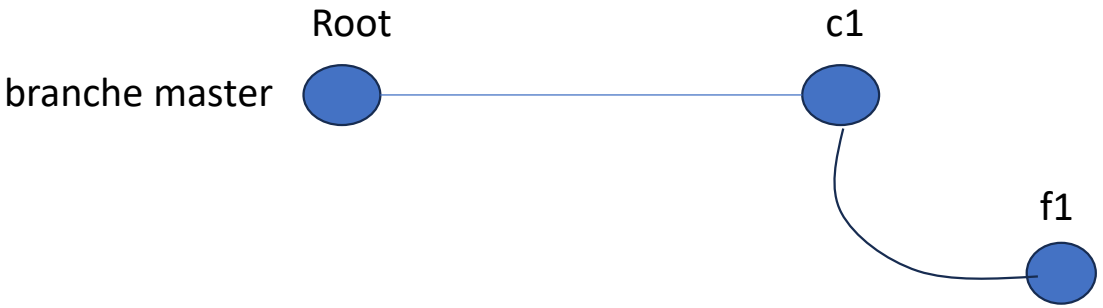
12_ créer la branche (façon 1) : git branch feature1
13_ saisir : git status → git dit que l'on se trouve sur main et rien n'a changé.
NOUS N AVONS PAS BASCULE SUR LA BRANCHE FEATURE1
14_ changeons de branche : git checkout feature1 → git répond qu'il a « switché » de branche !
15_ saisir : git status → git dit que l'on se trouve sur la branche feature1 et que rien n'est à faire
16_ JUSTE POUR TESTER UN RAJOUT DANS CETTE BRANCHE : modifier le contenu du fichier README:
rajouter donc : « # ceci est un readme feature1 »
17_ saisir : git status → git répond que l'on est sur feature1 et qu'il y a des modifs à enregistrer.
18_ mettons à jour l'index : git add ./README.md
19_ réalisons le commit : git commit -m « rajout de feature1 »
20_ nous sommes sur feature1 et on y a réalisé un premier commit pour notre code isolé.

GIT le gestionnaire de code source

UNE BRANCHE = 1 copie protégée = 1 version du code

Suite Manipulation 2:

Notre git ressemble à :



VOUS ETES SUR LA BRANCHE f1 feature1

21_ comment revenir à la branche main ?? Saisir : git checkout feature1
22_ vérifions que vous êtes revenu en saisissant : cat README.md
La modification f1 a disparu !!!
23_ Retournez sur la branche feature1 : saisir : git checkout feature1

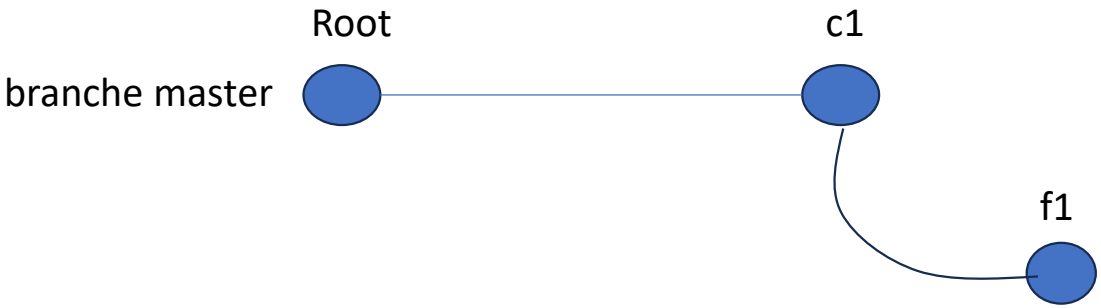
C EST LE CŒUR DE GIT: VOUS REVENEZ A D'AUTRES VERSIONS DE LOGICIELS

GIT le gestionnaire de code source

UNE BRANCHE = 1 copie protégée = 1 version du code

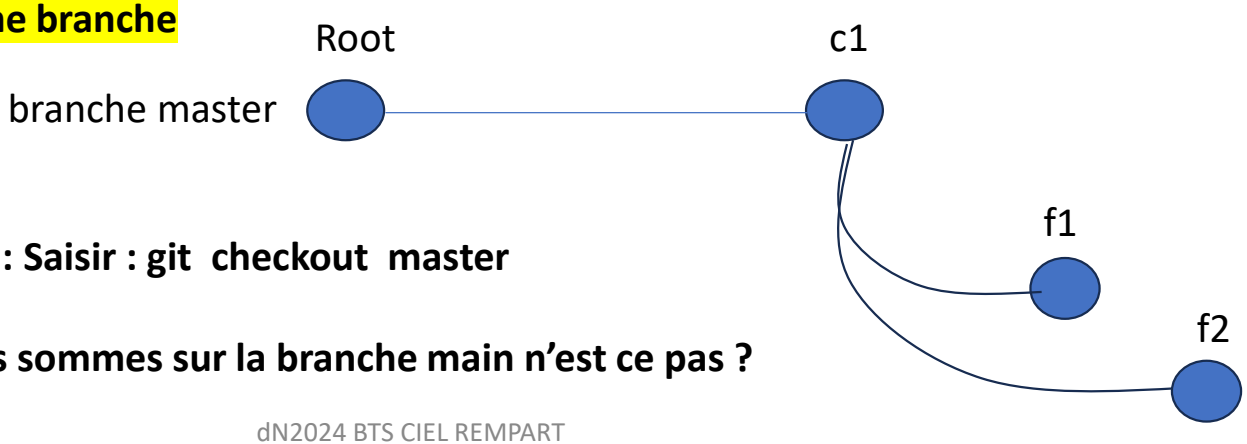
Manipulation 3:

Notre git ressemble à :



VOUS ETES SUR LA BRANCHE f1 feature1

24_ nous allons créer une deuxième branche



Revenir d'abord sur le master : Saisir : git checkout master

Git devrait nous dire que nous sommes sur la branche main n'est ce pas ?

GIT le gestionnaire de code source

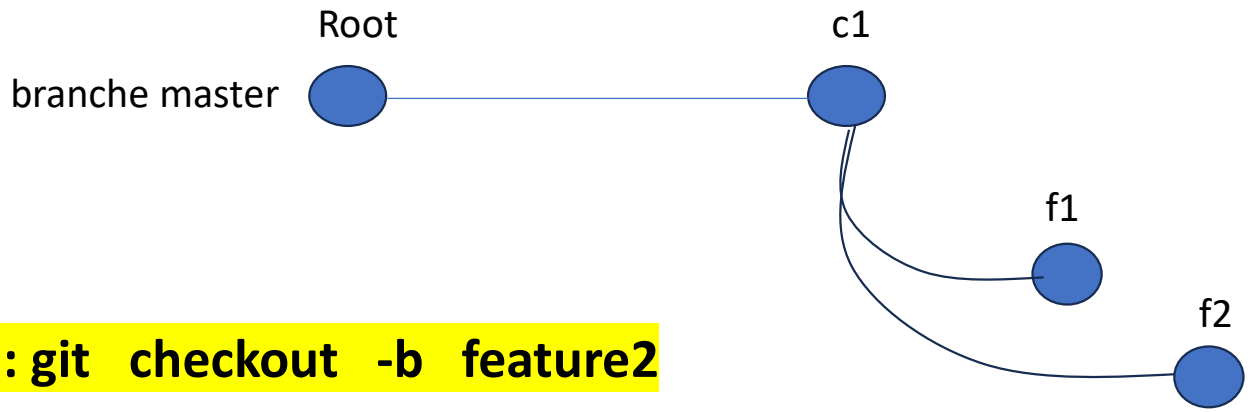
UNE BRANCHE = 1 copie protégée = 1 version du code

Manipulation 3:

Notre git ressemble à :

VOUS ETES revenu SUR LA BRANCHE main

25_ une fois sur le master ou branche main, nous allons créer une deuxième branche



Saisir : git checkout -b feature2

En saisissant : git status → <u>nous sommes bien allé sur la branche feature2</u>

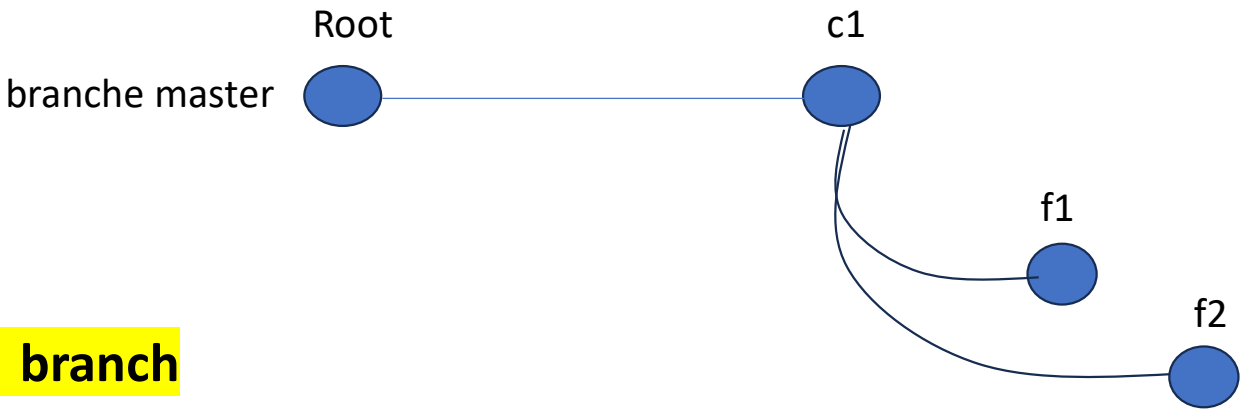
GIT le gestionnaire de code source

UNE BRANCHE = 1 copie protégée = 1 version du code

Manipulation 3:

VOUS ETES revenu SUR LA BRANCHE feature2

26_ bilan de toutes les branches (intérêt pédagogique uniquement ici):



Saisir : git branch

Il y a affichage de toutes les branches, la branche active est précédée d'un astérisque

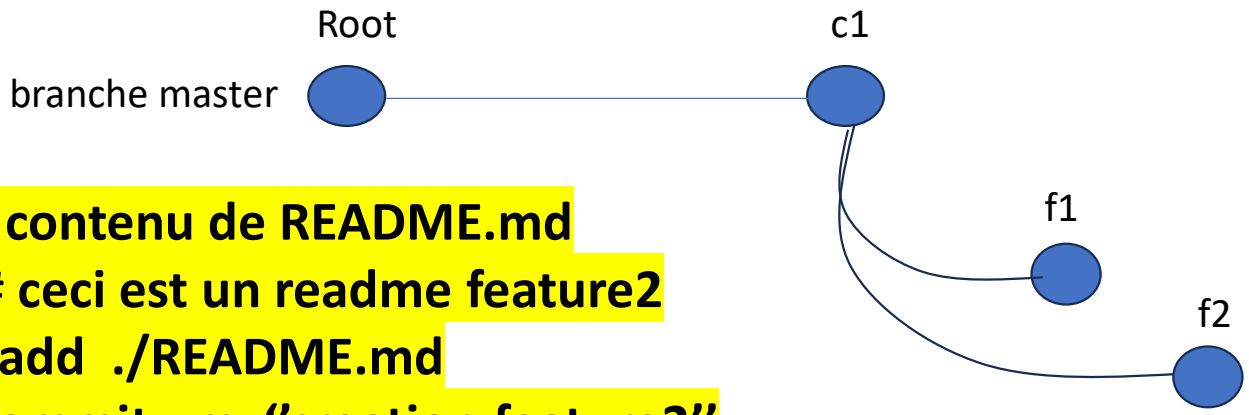
GIT le gestionnaire de code source

UNE BRANCHE = 1 copie protégée = 1 version du code

Manipulation 3:

VOUS ETES SUR LA BRANCHE feature2

27_ faire un commit sur feature 2:



Modifier le contenu de README.md
Rajouter : # ceci est un readme feature2
Saisir : git add ./README.md
Saisir : git commit -m "creation feature2"

GIT le gestionnaire de code source

UNE BRANCHE = 1 copie protégée = 1 version du code

SYNTHESE 1:

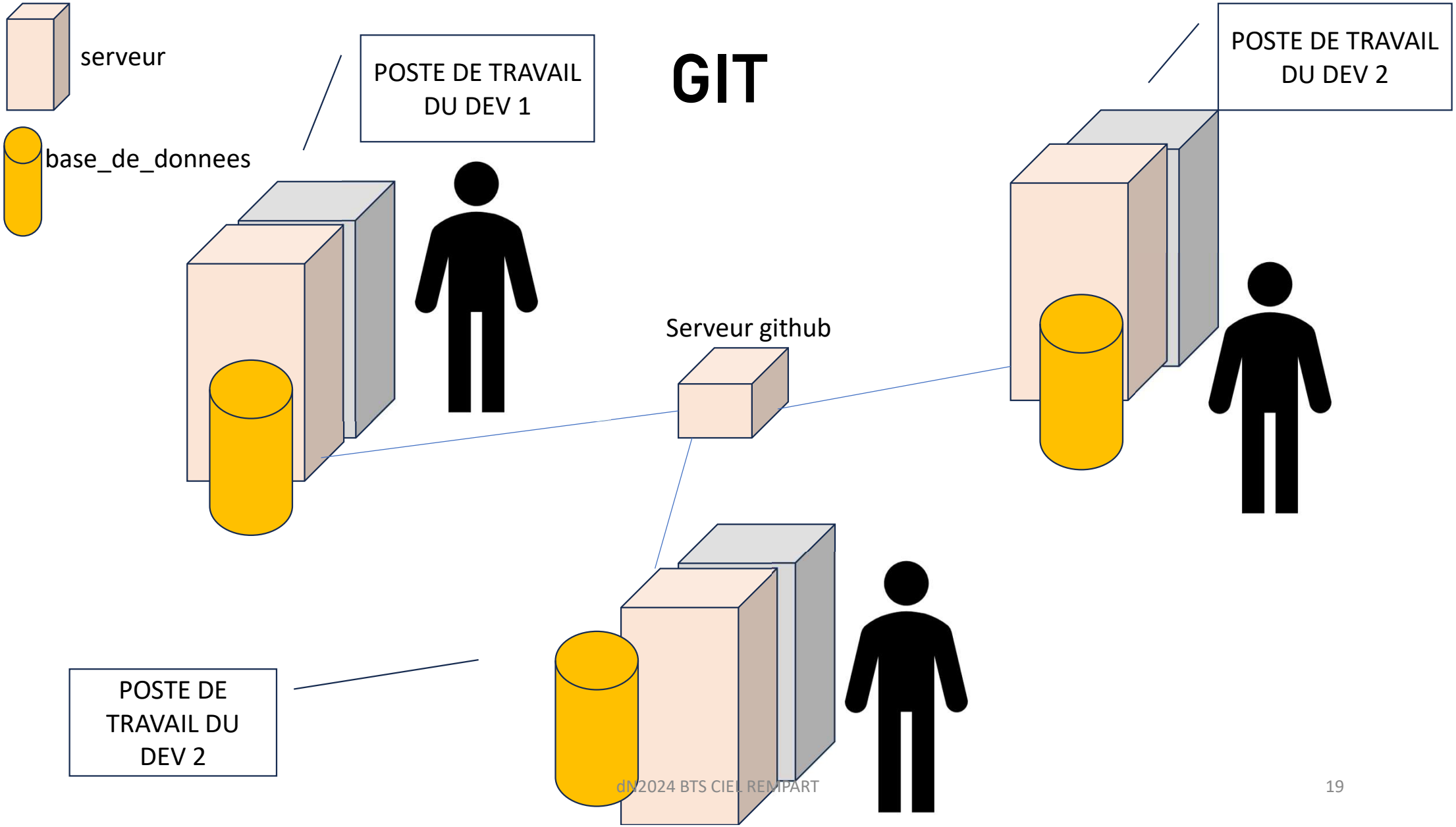
NOUS AVONS VU COMMENT :

- Créer une branche
- Changer de branche
- Créer et changer de branche en même temps avec ***git checkout –b <nomdelabranche>***
- Lister les branches avec git branch

GIT le gestionnaire de code source

PARTIE 2 : plusieurs développeurs travaillent avec GIT

GIT

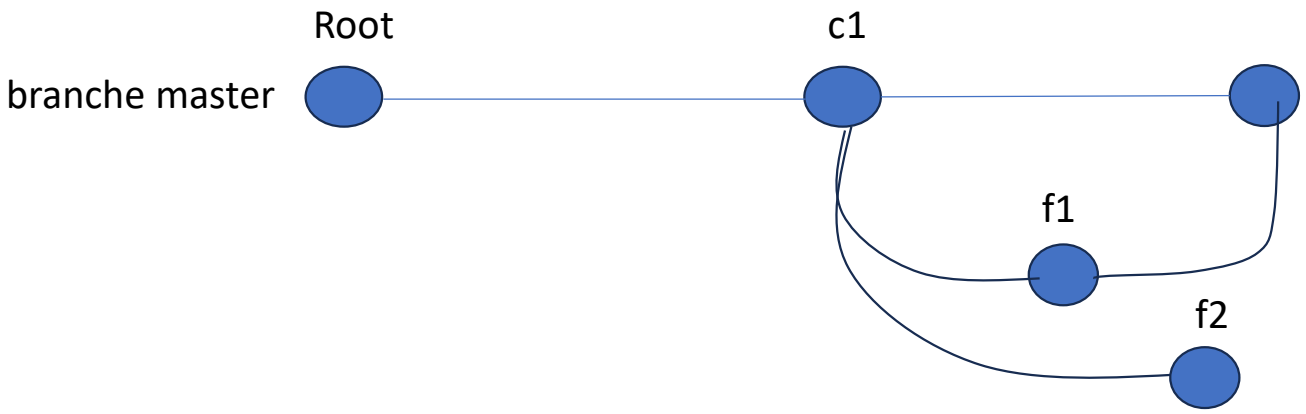

serveur
base_de_donnees
POSTE DE TRAVAIL DU DEV 1
POSTE DE TRAVAIL DU DEV 2
Serveur github
POSTE DE TRAVAIL DU DEV 2

GIT le gestionnaire de code source

28_ pour être sûr d'être sur la branche main : saisir : **git checkout main**

29_ **nous allons récupérer le contenu de la branche feature1 pour l'inclure sur la branche main.**
Pour cela, nous allons combiner (ou « merger »(verbe anglais)) la branche feature1 vers la branche main.
La commande git se nomme « merge », on lui indique la branche que l'on veut combiner sur la branche acturelle ou l'on se trouve.
Saisir : **git merge feature1** → git indique « updating…. » « README.md 1 file changed 1 insertion »

GIT le gestionnaire de code source



30_ ouvrir sous éditeur le fichier README.md c'est le contenu de feature1 !!!

31_ DEUX DEVELOPPEURS TRAVAILLENT SUR UN MEME FICHIER:

<u>ILS doivent posséder un compte distant github</u>

GIT le gestionnaire de code source

Créer un compte distant github

Suivre les instructions de : https://github.com/signup

Tutoriel : https://www.youtube.com/watch?v=Gn3w1UvTx0A

Configurer votre poste de travail pour qu'il communique bien avec github :

Si le dossier « .ssh » dans /home/etudiant n'existe pas … il faut le créer :
sudo mkdir .ssh

https://www.youtube.com/watch?v=6tkZmHNshtM

GIT le gestionnaire de code source

<u>Créer un nouvelle clef ssh :</u>

Se déplacer dans .ssh puis saisir :

ssh-keygen -t ed25519 -C « votre@email.com »

~~OU si non OK :~~

~~ssh-keygen -t rsa -b 4096 -C « votre@email.com »~~

Réponse : generating public/private algorithm key pair

GITHUB ATTEND VOTRE CLE PUBLIQUE
NE PAS SAISIR DE MOT DE PASSE

GIT le gestionnaire de code source

Informer linux que la clé ssh est générée :

Saisir et lancer l'agent (software) ssh :

sudo eval ''$(ssh-agent -s) ''

Saisir ensuite l'ajout ''à linux'' de la clé générée :

Si l'ed25519 a fonctionné : sudo ssh-add ~/.ssh/id_ed25519

Sinon : sudo … ?

GIT le gestionnaire de code source

Placer sur github la clé publique ssh générée :

Copier le contenu du fichier nomdelacle.pub dans la répertoire .ssh

Coller le contenu dans votre compte github